

# 1. Interfaz de Usuario (Frontend)

## 1.1. Diseño Visual: Estilo Neomorfista

Para la capa de presentación se decidió abandonar el paradigma clásico de *flat design* y adoptar un enfoque basado en **Neomorfismo**. Este estilo visual se implementa mediante reglas CSS que combinan múltiples sombras (**box-shadow**) con distintos niveles de opacidad y desplazamiento.

Técnicamente, cada componente gráfico se modela como una superficie pseudo-tridimensional generada por:

- Una sombra clara desplazada hacia la esquina superior izquierda.
- Una sombra oscura desplazada hacia la esquina inferior derecha.

Este patrón genera un efecto de relieve suave que simula la extrusión del elemento desde el fondo. Desde el punto de vista de experiencia de usuario (UX), el neomorfismo reduce el contraste agresivo propio de otros estilos, favoreciendo una interfaz visualmente continua y menos fatigante para el operador humano, particularmente en escenarios de uso prolongado.

Además, este enfoque se integra con principios de diseño responsivo mediante el uso de unidades relativas (**rem**, **%**, **vh**) y reglas **media queries**, garantizando consistencia visual en distintos tamaños de pantalla.

## 1.2. Consumo de API: Comunicación Asíncrona con Fetch

La aplicación adopta un modelo lógico de tipo **Single Page Application (SPA)** en cuanto a interacción con datos. Esto significa que la carga inicial del documento HTML es única y las posteriores actualizaciones de información se realizan mediante solicitudes asíncronas al servidor.

El mecanismo principal de comunicación se implementa mediante la función **fetch()** del estándar ECMAScript, la cual permite:

- Enviar peticiones HTTP (**GET**, **POST**, **PUT**, **DELETE**).
- Procesar respuestas en formato JSON.
- Evitar la recarga completa del documento.

Desde el punto de vista arquitectónico, esto desacopla la capa de presentación del backend, siguiendo el principio REST de separación cliente-servidor. Las actualizaciones de la interfaz se realizan manipulando directamente el **DOM (Document Object Model)**, lo cual permite:

- Renderizado dinámico de tablas y formularios.
- Validaciones previas al envío de datos.

- Reducción de latencia percibida por el usuario.

Este enfoque mejora la eficiencia computacional al minimizar la transferencia de HTML redundante y reduce la carga en el servidor, ya que se intercambian únicamente estructuras de datos.

### 1.3. Gestión de Estado Local: Persistencia de Sesión

La persistencia de sesión se gestiona mediante el uso del objeto `localStorage`, que actúa como un almacén clave-valor en el navegador del cliente.

Durante el proceso de autenticación:

- El backend emite un **JSON Web Token (JWT)**.
- El frontend almacena dicho token junto con el rol del usuario.

Esta estrategia permite mantener el estado de autenticación incluso tras una recarga del navegador. Desde el punto de vista técnico, se logra una persistencia sin necesidad de cookies tradicionales, reduciendo riesgos asociados a ataques CSRF cuando se configura correctamente el envío de cabeceras **Authorization**.

Adicionalmente, el rol almacenado se emplea para realizar control de acceso preliminar en la interfaz, permitiendo:

- Renderizar u ocultar botones según privilegios.
- Evitar interacciones inválidas desde la UI.

Cabe destacar que este control es complementario, ya que la validación definitiva ocurre siempre en el servidor.

## 2. Seguridad y Control de Acceso (Backend)

### 2.1. Encriptación: Protección de Credenciales

En el backend se implementa un mecanismo de protección de contraseñas mediante la librería `passlib` utilizando el algoritmo **bcrypt**.

Bcrypt introduce dos propiedades fundamentales:

- **Salting automático**: inserta valores aleatorios en cada hash, evitando ataques por tablas arcoíris.
- **Cost factor**: incrementa el tiempo de cómputo por intento, mitigando ataques de fuerza bruta.

Desde una perspectiva criptográfica, las contraseñas no se almacenan como texto plano sino como funciones hash unidireccionales, lo que implica que no existe una operación inversa para recuperar la contraseña original.

Esto garantiza que, incluso ante una intrusión a la base de datos, la información sensible permanezca protegida a nivel matemático.

## 2.2. Tokens JWT: Autenticación Stateless

El sistema emplea **JSON Web Tokens** para implementar autenticación sin estado (*stateless authentication*). Cada token encapsula:

- Identidad del usuario.
- Rol asignado.
- Marca temporal de expiración.

El servidor no mantiene una tabla de sesiones activas. En su lugar, cada solicitud entrante debe presentar un token válido en la cabecera HTTP. El backend verifica:

- La firma criptográfica.
- La vigencia temporal.
- La integridad del contenido.

Este enfoque mejora la escalabilidad horizontal, ya que múltiples instancias del servidor pueden validar tokens sin sincronizar estados internos.

La expiración de 60 minutos reduce la superficie de ataque en caso de interceptación del token.

## 2.3. Control de Acceso Basado en Roles (RBAC)

El modelo de autorización se implementa mediante **Role-Based Access Control (RBAC)**. Cada usuario posee un rol lógico definido en el sistema, como:

- Director.
- Coordinador.
- Usuario estándar.

En cada endpoint protegido, el backend inspecciona el rol contenido en el JWT antes de ejecutar la operación solicitada. Este proceso se realiza mediante middleware de validación.

Si el rol no cumple con los privilegios requeridos:

- La operación se cancela.

- Se devuelve un código HTTP 403 (*Forbidden*).

Este mecanismo garantiza que la jerarquía organizacional se refleje directamente en la capa lógica de datos, evitando dependencias exclusivas del frontend.

### 3. Decisión Tecnológica: Base de Datos

Se seleccionó PostgreSQL como sistema gestor de base de datos debido a sus capacidades avanzadas en:

- Control de concurrencia mediante MVCC.
- Integridad referencial con claves foráneas.
- Transacciones ACID.

A diferencia de motores embebidos como SQLite, PostgreSQL permite soportar múltiples usuarios concurrentes sin riesgo de corrupción de datos, lo cual resulta esencial en un entorno institucional con múltiples operadores simultáneos.

### 4. Conclusión

La arquitectura implementada integra principios modernos de desarrollo web:

- Separación de responsabilidades.
- Seguridad por capas.
- Escalabilidad lógica.

El sistema no solo cumple con requisitos funcionales, sino que incorpora fundamentos sólidos de ingeniería de software orientados a producción.